MSc thesis ▪ Hochschule Heilbronn

# Can an internal “debate” add-on make a small language model reason better?

## Progress report on the L2 add-on (project Ekdant)

Harshil Gorasiya

Status: 13 November 2026 – revised after the supervisor’s comments

**In one paragraph**

L2 is a 19.6 M-parameter add-on attached to the pretrained language model SmolLM2-135M. It runs four internal candidate states (“trajectories”) of each token, lets them debate and averages them. Over nine L2 versions, three training seeds per final model and three purpose-built reasoning tests, **L2 never showed better reasoning**. Its best accuracy gain (+2.07* points on PubMedQA) turned out to be a preference for answering “yes”. The positive results of the project are a **data finding** (training the yes/no decision from the evidence: PubMedQA 58.0 % → 61.3 % without any add-on) and an **evaluation method** that separates reasoning from answer preference.

All numbers are means over seeds 17, 23 and 31. Differences carry a paired bootstrap 95 % confidence interval; * marks an interval that excludes 0. Accuracy differences are in percentage points.

# Contents

**1. The question** **3**

**2. Working hypothesis** **4**
2.1. The idea in one sentence . . . 4
2.2. Why each part of L2 should help (the assumed mechanism) . . . 4
2.3. Testable predictions and what happened . . . 4

**3. Related work** **5**
3.1. Multi-path reasoning in text . . . 5
3.2. Reasoning in latent space . . . 5
3.3. Several streams inside one network . . . 6
3.4. Energy-based refinement and collapse . . . 6
3.5. Calibration, answer bias and counterfactual data . . . 6
3.6. Medical question answering . . . 6

**4. The model** **6**
4.1. The backbone: SmolLM2-135M . . . 6
4.2. Where L2 attaches . . . 7

**5. The L2 layer in detail** **7**
5.1. L2 in plain words: a committee of four . . . 7
5.2. Step 1 – four viewpoints (diverse start) . . . 8
5.3. Step 2 – discussion, four rounds (debate) . . . 9
5.4. Step 3 – staying grounded (anchored descent) . . . 9
5.5. Step 4 – the verdict and the gate . . . 10
5.6. A safety rule during training: the spread floor . . . 11
5.7. The whole layer in one technical picture . . . 11
5.8. Parameter count . . . 12
5.9. What L2 can and cannot see . . . 12

**6. Data** **13**
6.1. Datasets . . . 13
6.2. The "old data" (v1–v5, T59, T60) . . . 13
6.3. The evidence data (T61, `--evidence`) . . . 13
6.4. The pair loss (T63, `--pair-loss 1.0`) . . . 14

**7. Training** **14**

**8. Evaluation** **14**
8.1. Multiple-choice accuracy . . . 14
8.2. Text measures . . . 15
8.3. The three reasoning tests (T59) . . . 15
8.4. Statistics and the L2-off test . . . 15

**9. Experimental history** **16**

**10. Results** **17**
10.1. PubMedQA accuracy . . . 17
10.2. Reasoning tests . . . 18
10.3. What L2 adds on the same data . . . 19

10.4. Switching L2 off at test time . . . 19
10.5. Answer preference . . . 20
10.6. Text generation . . . 21

**11. Discussion** **21**
11.1. Verdict on L2 . . . 21
11.2. A likely reason (interpretation) . . . 22
11.3. The "yes" preference is a calibration effect . . . 22
11.4. Only one backbone: would a larger model change the picture? . . . 24
11.5. Limitations . . . 24

**12. Conclusions** **25**

**13. Supervisor's comments and points for the meeting** **25**

**A. Glossary** **27**

**B. Reproducibility** **27**

**Findings at a glance**

| | |
|---|---|
| **Worked** | **Training the decision from the evidence (T61).** PubMedQA 58.0 → 61.3 %* without any add-on; following flipped evidence 27 % → 79–80 %; 3-benchmark average +1.25*; a fifth of the training text. |
| **Worked** | **Three reasoning tests + L2-off test.** They exposed L2's +2-point PubMedQA gain as a "yes" preference (+11.0* more "yes" answers), which accuracy alone hides. |
| **Not useful** | **The L2 layer.** Reasoning never shown; with the right data it adds −0.10 points; text perplexity drops by under 2 % but the generated text is no better. |

# 1. The question

Small language models answer medical questions largely by pattern matching. The idea behind this thesis is to add a separate "thinking" module on top of an ordinary language model. Inside it, several candidate versions of the model's internal state – *trajectories* – start from different angles, read each other, revise themselves and agree on one answer state: a small internal debate inside a single forward pass, without generating extra text.

The original design had two parts, L1 and L2. L1 is left for future work; this thesis tests **L2 alone**, on a pretrained model, against the same model without it.

**Research question**

Does SmolLM2 + L2 *reason* better on medical question answering than SmolLM2 alone, when both are trained on exactly the same data? "Reason better" must mean more than higher accuracy; Section 8 defines how it is tested.

# 2. Working hypothesis

## 2.1. The idea in one sentence

A language model commits to *one* internal state per token and predicts from it. Methods that make large models reason better – self-consistency, multi-agent debate – work by producing *several* candidate answers, letting them check each other, and reconciling them; but they do this in text, at $K$ times the inference cost (Section 3).

**Working hypothesis (H)**

The benefit of "consider several alternatives, cross-check them, reconcile them" can be obtained *inside one forward pass, at the level of hidden states.* If a token's final state is expanded into $K$ diverse hypotheses that read and revise each other and are then reconciled towards an anchor derived from the evidence-bearing state, the resulting consensus state supports a decision that depends *more on the evidence* and *less on surface priors* than the single state – at a fixed cost of one extra module, with no extra tokens.

## 2.2. Why each part of L2 should help (the assumed mechanism)

1. **Diversity** (diverse start): $K$ different projections of the same state emphasise different aspects of it, like several readers of the same abstract. Without diversity there is nothing to reconcile.
2. **Error correction by cross-reading** (debate): a hypothesis that is wrong in one aspect can be corrected by the others; the repair gate lets a correct minority resist being overruled. This is the representation-level analogue of multi-agent debate [14].
3. **Grounding** (anchored descent): pulling every hypothesis towards an anchor computed from the backbone state keeps the debate tied to what was read and prevents drift into a self-consistent but wrong consensus.
4. **Aggregation** (average): averaging reduces the variance of individual hypotheses, as voting does in self-consistency [13].
5. **Honesty signal**: when the hypotheses disagree, the model should be less sure; trajectory agreement should therefore predict correctness.

## 2.3. Testable predictions and what happened

The hypothesis was written as falsifiable predictions before the decisive runs. The prior-art audit had already found a compute-matched study reporting *no reliable gain* from recursive latent refinement in small models [20], so a negative outcome was considered from the start.

| | If H is true … | Test | Outcome |
|---|---|---|---|
| P1 | accuracy rises on *both* yes- and no-questions | test 1 (Section 8) | **not met**: gain only on yes-questions; "yes" answers +11.0* |
| P2 | the answer depends more on the abstract | test 2 (unseen) | **not met**: +0.17 (old data), −1.07* (evidence data) |
| P3 | the answer follows flipped evidence more often | test 3 | **not met**: +3.25, +1.22, +1.02, never starred |
| P4 | L2 does the work at test time | L2-off test | partly: the model leans on L2, but is no better than without it |
| P5 | trajectory agreement predicts correctness better than the model's own probability | AUROC (v3c, T54) | **not met**: next word 0.81 vs 0.906; PubMedQA 0.52–0.53 vs 0.60–0.62 |

**Table 1:** Predictions of the working hypothesis. None of the reasoning predictions was met.

## 3. Related work

L2 combines ideas from five lines of work. A prior-art audit (vault note 29, papers stored locally) concluded that **every single ingredient has been implemented before; the combination in one independent module after a pretrained backbone has not**. The contribution claimed is therefore a combination, and the closest competitors are named below.

### 3.1. Multi-path reasoning in text

Chain-of-thought prompting [12] lets a model write intermediate steps; in L2's terms it is a single path ($K = 1$). Self-consistency [13] samples several reasoning paths and takes a majority vote – the direct inspiration for L2's aggregation. Multi-agent debate [14] lets several model instances read and revise each other's answers over rounds; L2's debate rounds are its blueprint. Divergent-thinking debate [15] shows that diverse starting points matter, which motivated the diverse start. Mixture-of-Agents [16] stacks layers of agents that read the previous layer's answers. A critical evaluation [17] finds that debate often does not beat self-consistency at equal cost. All of these operate on generated *text*; L2 moves the same pattern into hidden states.

### 3.2. Reasoning in latent space

Coconut [18] feeds the model's hidden state back as a "continuous thought" instead of a word. Tiny Recursive Models [19] reason by repeatedly refining a latent state with a very small network. **The most important related result is negative:** a compute-matched comparison of recursive latent models [20] found that the full autoregressive recursive model gives *no reliable gain* over simpler variants; only a flat two-stream split with a weighted readout helped. Our results agree with this finding.

### 3.3. Several streams inside one network

Multi-Stream Transformers [21] split a network into parallel streams merged at the end – the closest architectural ancestor of L2. Talking-Heads Attention [22] lets attention heads depend on each other, comparable to trajectories reading each other. The Dual-Stream Transformer [23] separates the residual stream into functional parts and is evaluated at a similar small scale (29 M parameters). L2 differs in that its streams are full-width copies of a token's state that cross-read, are anchored and are averaged, after a pretrained backbone.

### 3.4. Energy-based refinement and collapse

The Energy Transformer [24] computes its output by descending an energy function for a fixed number of steps; L2's anchored descent is a minimal version of this. Work on agreement objectives shows they tend to collapse diversity [25]; we found the same defect in an early L2 version and replaced the agreement loss by the spread floor. Internalising debate into one model has also been done by *distillation* [26]; L2 does it architecturally, without a teacher.

### 3.5. Calibration, answer bias and counterfactual data

Neural networks are often miscalibrated [27], and language models show a *majority-label bias*: they over-predict answers that were frequent in their training or prompt, which can be corrected by shifting the output log-odds ("contextual calibration") [28]. This is the lens for the "yes" preference in Section 11.3. Counterfactually augmented data [7] – minimally edited examples with flipped labels – teach a model which part of the input decides the label; PairCFR [6] adds a loss over both rows of a pair. These are the basis of T61 and T63.

### 3.6. Medical question answering

PubMedQA [2], MedMCQA [3], MedQA [4] and MedQuAD [5] are the standard public benchmarks used here; scoring follows lm-evaluation-harness [8]. Small models such as SmolLM2-135M [1] stay near chance on the exam-style sets, which limits what they can show (Section 11.4).

## 4. The model

### 4.1. The backbone: SmolLM2-135M

SmolLM2-135M [1] is a public, pretrained, decoder-only language model whose task is to predict the next token. All of its weights are fine-tuned (full fine-tuning); LoRA was tried only as one variant (T60).

| Property | Value |
|---|---|
| Parameters | 134,515,584 (≈ 135 M) |
| Layers | 30 decoder layers (self-attention + MLP), RMSNorm [10], rotary position encoding |
| Hidden size | 576 numbers per token – the token's *state* $h$ |
| Attention | 9 query heads, 3 key/value heads (grouped-query attention) |
| Vocabulary | 49,152 tokens; output layer (`lm_head`) tied to the input embedding |
| Context used | up to 1,024 tokens per example (answers up to 512) |

**Table 2:** The backbone.

### 4.2. Where L2 attaches

In the final model (v5), L2 sits between SmolLM2's last layer and its output layer (Figure 1). For every token, SmolLM2 produces a final state $h \in \mathbb{R}^{576}$. L2 computes a refined state $y$, and a learned per-channel gate $g$ decides how much of $y$ replaces $h$:

$$h' = h + g \odot (y - h), \qquad g = \tanh(\gamma_{\text{gate}}) \in \mathbb{R}^{576}, \quad \gamma_{\text{gate}} = 0 \text{ at the start.} \tag{1}$$

Because the gate starts at 0, the model at step 0 *is* SmolLM2; any difference between the two arms is learned. In trained v5 the gates end at $|g| \approx 0.025$–$0.028$, i.e. L2 replaces roughly 3 % of the final state.

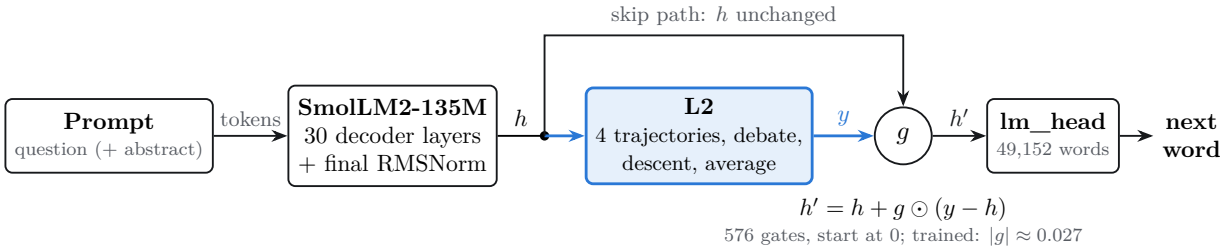



**Figure 1:** The whole model. SmolLM2 produces the state $h$ of every token. $h$ goes both into L2 and, unchanged, along the skip path. The gate mixes the two before the output layer predicts the next word. $y = \text{RMSNorm}(c)$, where $c$ is L2's consensus state (Figure 8).

## 5. The L2 layer in detail

This section first explains L2 in plain words, one step at a time with a picture for each step, and then gives the same steps as formulas. All settings are those of the final model v5 (`configs/v5_final.json`; code in `09-Code/src/ekdant/sandbox.py` and `model.py`).

### 5.1. L2 in plain words: a committee of four

Without L2, SmolLM2 forms *one* understanding of each word – its state $h$, a list of 576 numbers – and answers from it straight away. Nothing checks that understanding. L2 adds a small **committee of four reviewers** between SmolLM2's understanding and the answer: they look at the understanding from four angles, discuss it, are kept close to what the text said by a moderator,

and agree on a verdict. A dial (the gate) then decides how much of the verdict is used. All of this happens with numbers inside one forward pass; no extra text is written.

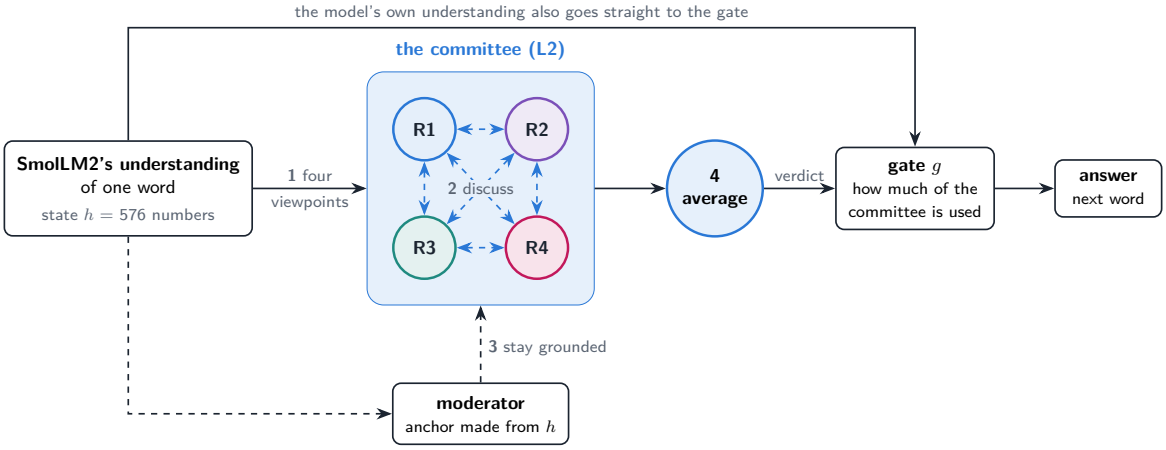



**Figure 2:** L2 as a committee. (1) four viewpoints of the same word, (2) four rounds of discussion, (3) a moderator keeps them grounded, (4) the verdict is the average, and the gate decides how much of it reaches the answer. The model's own understanding always reaches the gate too.

**One important limit**

L2 works **one word at a time** and only sees SmolLM2's final state of that word. It never reads the abstract itself: everything it knows is what SmolLM2 has already put into $h$.

## 5.2. Step 1 – four viewpoints (diverse start)

L2 makes four versions of the same state $h$, each turned to a different angle by its own learned transformation. They are called *trajectories* $T_1, \ldots, T_4$ – in the analogy, four reviewers who each focus on something different. They are built to start clearly different, because four identical reviewers would have nothing to discuss.

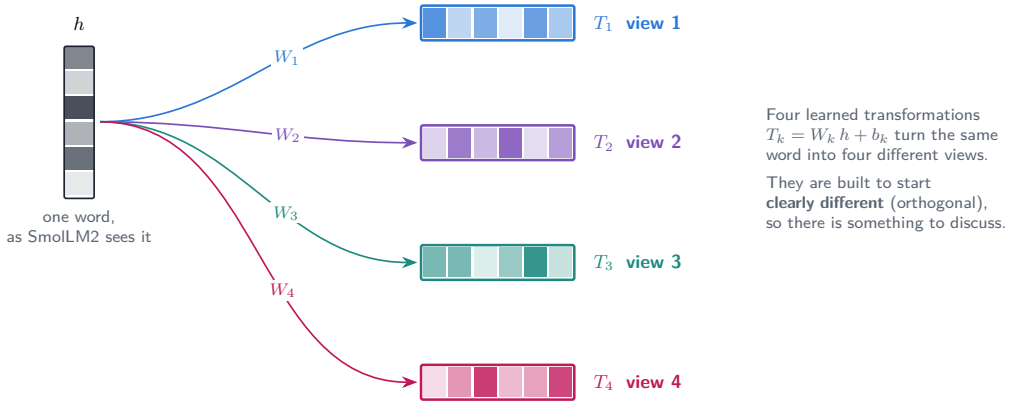



**Figure 3:** Step 1. The same word is turned into four different views.

*In formulas:* $T_k = W_k h + b_k$ for $k = 1, \ldots, 4$, where $W_1, \ldots, W_4 \in \mathbb{R}^{576\times576}$ start mutually orthogonal (Frobenius inner product 0) with gains spread from 0.8 to 1.25.

## 5.3. Step 2 – discussion, four rounds (debate)

In every round each reviewer (i) reads what the other three currently think – never itself, (ii) forms a proposal for how to update its view, and (iii) decides with a dial, the **repair gate** $\gamma$, how much of the proposal to accept: near 0 it keeps its own view, near 1 it takes the proposal. The dial starts cautious, at about 0.12. Its purpose is to let a reviewer who is right *resist the crowd*, so a correct minority can survive a round. Each of the four rounds has its own weights.

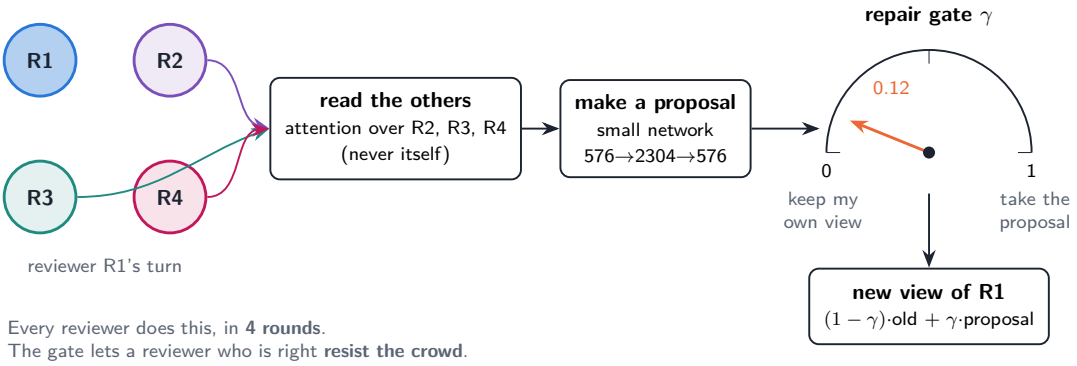



**Figure 4:** Step 2, one reviewer's turn in one round. All four reviewers do this in each of the four rounds.

*In formulas* (round $r = 1, \ldots, 4$):

$$x_k = \mathrm{RMSNorm}(T_k), \tag{2}$$

$$\mathrm{cross}_k = \sum_{j\neq k} \mathrm{softmax}_j\left(\frac{(Qx_k)^\top (Kx_j)}{\sqrt{576}}\right) Vx_j, \tag{3}$$

$$\gamma_k = \sigma\big(W_\gamma[x_k; \mathrm{cross}_k] + b_\gamma\big), \qquad b_\gamma = -2 \text{ at the start}, \tag{4}$$

$$T_k \leftarrow (1-\gamma_k)\odot T_k + \gamma_k \odot \mathrm{MLP}(\mathrm{cross}_k). \tag{5}$$

The attention runs across the four trajectories of the same word, never across words.

## 5.4. Step 3 – staying grounded (anchored descent)

A discussion can drift away from the facts. L2 therefore computes an **anchor** $a$ from the original state $h$ – a reference point for "what the text said" – and moves every reviewer a little towards it, in 8 small steps. Each reviewer has its own pull strength (a second learned dial), so they move by different amounts and do not collapse into one point. In the analogy, this is the moderator saying "remember what the abstract actually says".

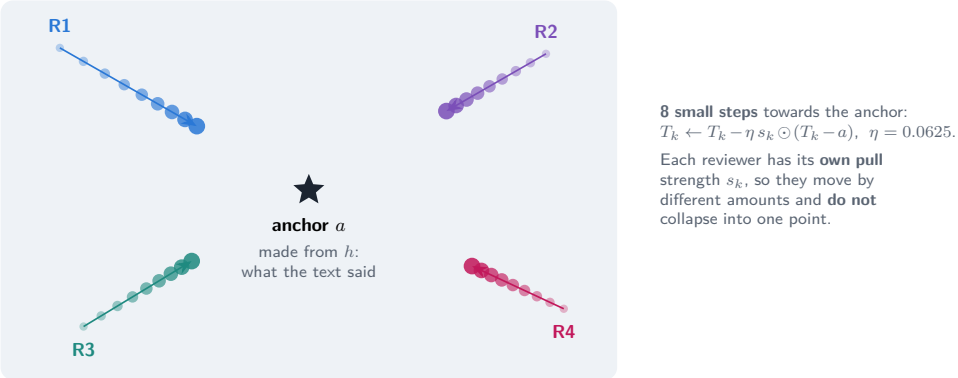



**Figure 5:** Step 3. Each reviewer takes 8 small steps towards the anchor, each with its own pull strength.

*In formulas* (8 steps, $\eta = 0.0625$): $a = W_a h$ and $T_k \leftarrow T_k - \eta\, \sigma\big(W_s[T_k; a]\big) \odot (T_k - a)$. The anchor must come from outside the committee: pulling the reviewers towards their own average provably changes nothing (vault note 29).

## 5.5. Step 4 – the verdict and the gate

The four views are averaged into one **verdict** $c$. The model does not simply replace its own understanding with it: the **gate** $g$ (one dial per number, 576 in total) decides how much of the verdict is mixed in. The gate starts at 0, so at the start of training the model is exactly SmolLM2 and every difference is learned. In the final model it ended at $g \approx 0.027$: the committee changes about 3 % of the final state.

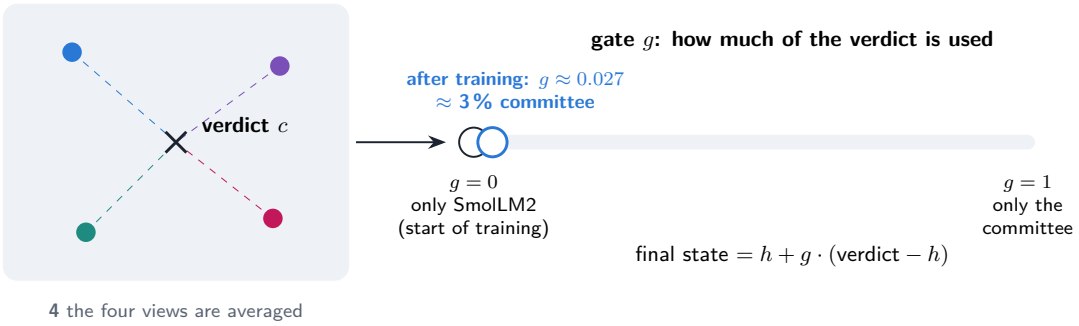



**Figure 6:** Step 4. The verdict is the average of the four views; the gate decides how much of it reaches the answer.

*In formulas:*

$$c = \frac{1}{4}\sum_{k=1}^{4} T_k, \qquad y = \mathrm{RMSNorm}(c), \qquad h' = h + g \odot (y - h), \qquad \mathrm{logits} = \mathrm{lm_head}(h'). \quad (6)$$

## 5.6. A safety rule during training: the spread floor

If the four reviewers became identical, the committee would be pointless – L2 would be one ordinary network. Training therefore adds a penalty, but *only* when the views come too close together; as long as they stay apart it costs nothing.

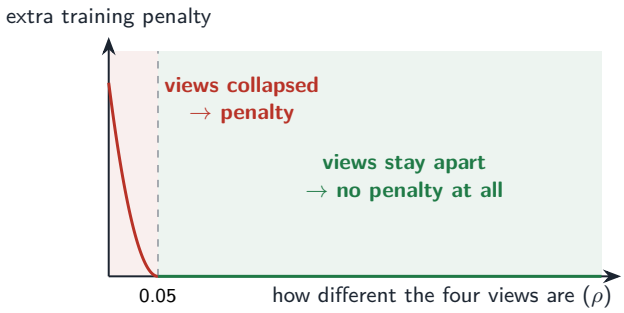



**Figure 7:** The spread floor: no penalty while the four views stay apart, a growing penalty once they collapse.

*In formulas:* $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \max(0,\ \rho_{\min} - \rho)^2$ with $\rho = \text{Var}_k(T_k)/\overline{h^2}$, $\rho_{\min} = 0.05$, $\lambda = 1$.

## 5.7. The whole layer in one technical picture

Figures 8 and 9 show the same four steps as they are wired in the code.

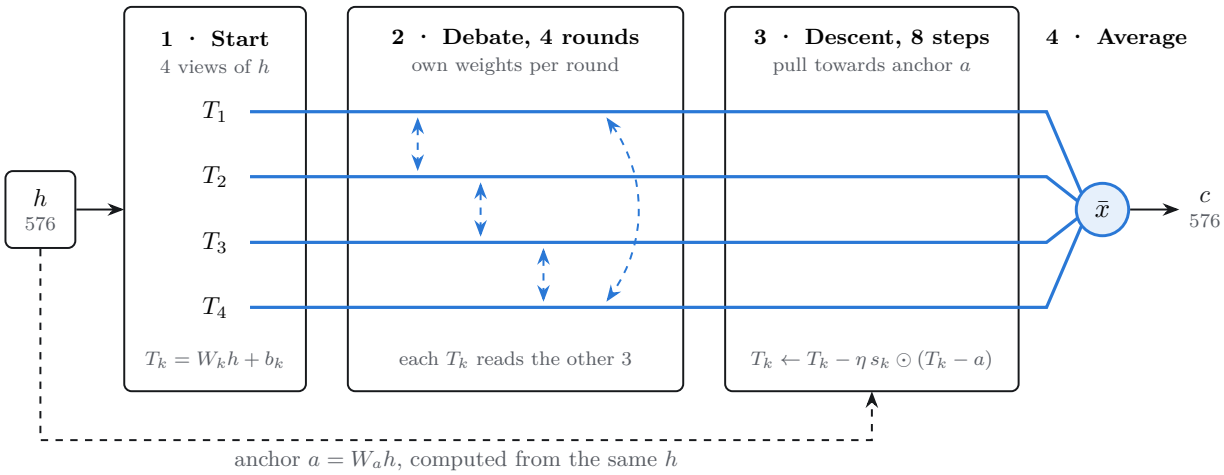



**Figure 8:** Inside L2, for one token. Blue lines are the four trajectories; dashed blue arrows are the cross-reads in every debate round; the dashed black line is the anchor, computed from the same input $h$. The output $c$ is L2's consensus state.

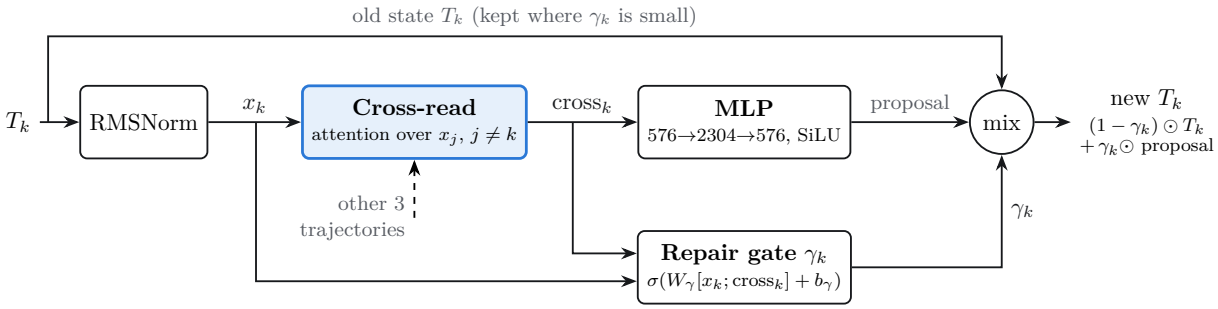



**Figure 9:** One debate round, for trajectory $k$. The repair gate $\gamma_k$ lets a trajectory keep its own state instead of following the others. Its bias starts at $-2$, so initially $\gamma_k \approx 0.12$ and each round changes little.

## 5.8. Parameter count

| Part | Weights | Parameters |
|---|---|---|
| Diverse start | 4 matrices $576 \times 576$ + 4 biases | 1,329,408 |
| One debate round | pre-norm 576; $Q, K, V$: $3 \times 576^2$; repair gate $1152 \to 576$; MLP $576 \to 2304 \to 576$ | 4,317,120 |
| 4 debate rounds | 4× the above | 17,268,480 |
| Anchored descent | anchor $576^2$; descent gate $1152 \to 576$ | 995,904 |
| Output | output RMSNorm 576; gate 576 | 1,152 |
| **L2 total** | | **19,594,944** (+14.6 %) |
| SmolLM2 + L2 | | 154,110,528 |

**Table 3:** Parameters of L2 in v5. The comparison with the baseline is architecture-matched, not parameter-matched.

## 5.9. What L2 can and cannot see

**An important property of the final design**

In v5, L2 works **per token**: it sees only that token's final state $h$, not the other tokens and not the abstract itself. Everything it knows about the question and the evidence is what SmolLM2 has already encoded in $h$. Because L2 sits at the very end, no later layer reads its output either. L2 can therefore only *reshape* what SmolLM2 has computed; it cannot gather new evidence. Later versions (v3–v4) gave L2 its own attention over the text, a verifier, a vote and placements inside the network; none did better than v5 (Section 9).

# 6. Data

## 6.1. Datasets

| Dataset | Content | Use |
|---|---|---|
| PubMedQA `pqa_artificial` [2] | 211,269 research questions, each with a PubMed abstract, the authors' conclusion and an automatically generated yes/no label | Training, stage 1 (2,000 rows held out) |
| MedQuAD [5] | 16,407 consumer-health questions with answers from NIH websites (14,345 unique questions) | Split by unique question: one half trains stage 2 (10 % of it selects the checkpoint), the other half is only a benchmark |
| PubMedQA `pqa_labeled` [2] | 1,000 questions with expert yes/no/maybe labels | Benchmark only; no PubMed id shared with the training set (checked) |

**Table 4:** Training and evaluation data. All datasets are public.

## 6.2. The "old data" (v1–v5, T59, T60)

Stage 1 was trained as **question + abstract → write the authors' conclusion** (209,269 rows, about 85 M tokens per epoch). Two problems were found:

1. **The labels are 92.8 % "yes"**: 196,144 yes, 15,125 no, no maybe.
2. **The yes/no word was never a training target.** The model only learned to write the conclusion, whereas the PubMedQA benchmark scores exactly the yes/no word. The benchmark answer was therefore a side effect of conclusion writing.

This explains how L2 could raise PubMedQA accuracy by learning to say "yes" more often – the majority answer – without reading the evidence any better.

## 6.3. The evidence data (T61, `--evidence`)

Only stage 1 is rebuilt; MedQuAD and every benchmark stay unchanged. The training answer now starts with the decision, and half of the rows force that decision to come from the evidence. The idea follows counterfactually augmented data [7].

| Rows | Prompt | Target | Count (seed 17) |
|---|---|---|---|
| Balanced | question + abstract | `yes. <conclusion>` or `no. <conclusion>` | 29,934 |
| Flipped pairs | question + the conclusion only | `yes` / `no` | $2 \times 45{,}774$ |

**Table 5:** The evidence data: 121,482 short rows, about a fifth of the old text (about 0.9 h per training run).

- **Balanced rows:** every "no" row plus the same number of "yes" rows, so always answering "yes" scores only 50 %.

- **Flipped pairs:** each conclusion appears twice – once as written with its real label, and once with one negation added or removed (e.g. "does not improve" ↔ "does improve") with the opposite label. Only a model that actually reads the sentence gets both right.
- The 1,000 benchmark questions are excluded from training.

### 6.4. The pair loss (T63, `--pair-loss 1.0`)

With plain cross-entropy, each row of a flipped pair is trained on its own. PairCFR [6] reports that counterfactual pairs help more when the loss compares both rows. Both rows of a pair are therefore placed in the same batch, and the loss adds

$$\mathcal{L}_{\text{pair}} = W \cdot \text{softplus}\left(m_{\text{no-row}} - m_{\text{yes-row}}\right), \qquad m = \text{logit}(\text{“yes”}) - \text{logit}(\text{“no”}), \quad W = 1, \quad (7)$$

with $m$ taken at the first answer token. $\mathcal{L}_{\text{pair}} \approx 0$ when the answer flips with the evidence and $\log 2 \approx 0.69$ when both versions get the same answer. It reuses the logits of the normal forward pass, so it costs no extra compute. Both arms receive it.

## 7. Training

| Item | Setting |
|---|---|
| Stage 1 | PubMedQA (old data or evidence data), 1 epoch |
| Stage 2 | MedQuAD training half, question → answer, 3 epochs; better of best/last checkpoint on its 10 % slice |
| Batch | 8 sequences × 4 accumulation steps = 32 per update; bf16 on one NVIDIA A100 80 GB per run |
| Optimiser | AdamW; learning rate SmolLM2 $5 \cdot 10^{-5}$, L2 matrices $10^{-4}$, gates $10^{-3}$; cosine schedule per stage, 200 warm-up steps |
| Clipping | gradient norm 1.0, separately for SmolLM2 and for L2 |
| Loss | next-token cross-entropy on answer tokens + spread floor (L2 only) + pair loss (T63 only) |
| Fairness | identical data, data order, split and starting weights for both arms; seeds 17, 23, 31 |

**Table 6:** Training setup, identical for every model in this report.

## 8. Evaluation

### 8.1. Multiple-choice accuracy

| Benchmark | Content | Questions | Chance |
|---|---|---|---|
| PubMedQA (`pqa_labeled`) [2] | research question + abstract → yes/no/maybe; expert labels | 1,000 | 33 % |
| MedMCQA (validation) [3] | Indian medical entrance exams, 4 options | 4,183 | 25 % |
| MedQA-USMLE (test) [4] | US medical licensing exam, 4 options | 1,273 | 25 % |

**Table 7:** Multiple-choice benchmarks.

Scoring follows lm-evaluation-harness [8]: the question is asked in the training format (`Question: … Answer:`, plus `Context:` for PubMedQA), and the model's answer is the option with the highest mean log-probability per token. Nothing is generated, so no answer parsing is needed.

## 8.2. Text measures

- **Perplexity** of a reference text (lower = predicted better): the MedQuAD benchmark half, MedMCQA expert explanations, PubMedQA conclusions.
- **Generated text**: the model writes its own MedMCQA explanation or PubMedQA conclusion (greedy decoding, 200 per set), scored by ROUGE-L [9] and by the share of explanations that **name the right answer**.

## 8.3. The three reasoning tests (T59)

| Test | What it measures | Reasoning would show as |
|---|---|---|
| 1. Where the gain comes from | PubMedQA accuracy split by the true answer, and how often each answer is chosen | gains on yes *and* no questions, not only "says yes more often" |
| 2. Uses the abstract | % right with the abstract minus % right without it (same 1,000 questions); never trained on | a starred gain: the answer depends more on the evidence |
| 3. Follows flipped evidence | the authors' one-sentence conclusion as the only context, once as written and once with one negation flipped (164 questions); "right on both" counts | a starred gain in "right on both" |

**Table 8:** The three reasoning tests.

**Decision rule, fixed before the runs**

L2 "reasons better" only if tests 2 **and** 3 both show a starred gain over the same model without L2. From T61 on, flipped conclusions are part of the training data (on other questions), so test 3 is no longer unseen for those runs; test 2 stays unseen.

## 8.4. Statistics and the L2-off test

Every number is a mean over three seeds. Differences between two models come with a **paired bootstrap 95 % confidence interval** [11]: questions are resampled, the same resample is used in every seed, and both models are scored on the same questions. Accuracy differences are reported in percentage points.

In the **L2-off test**, every L2 model is scored again with L2 switched off ($g = 0$). If the score drops, the model depends on L2; if it stays, L2 is not doing the work at test time.

# 9. Experimental history

| Step | What was done | Outcome |
|---|---|---|
| v1 | 50 M-parameter model with L2, trained from scratch | learned only the most frequent answer letter → switch to a pretrained backbone |
| v2.0–v2.2 | L2 on SmolLM2: at the end (v2.0), after layer 20 (v2.1), at several depths (v2.2) | at the end works best; small perplexity gains |
| v3, v3b, v3c | L2 reads the text itself, starts at the model's own state; verifier, vote, confidence, per-token gate; v3c "thinks twice" | no better than v2.2 |
| v4 | L2 inside the network and at the end, reading earlier layers | worse than v2.2 (average $-0.49^*$) |
| **v5** | final L2 model = the v2.2 "end" recipe (Section 5), three seeds | PubMedQA $\mathbf{+2.07}^*$, average $+0.72^*$, MedQuAD perplexity $-0.16\,\%^*$ |
| v5drop, v5out | L2 switched off on 30 % of training steps (T57); L2 output at the model's scale (T58) | neither better than v5 |
| T59 | the three reasoning tests on v5 | a preference: "yes" $+11.0^*$; tests 2 and 3 not starred |
| T60 | train L2 first (warm-up), or LoRA instead of full fine-tuning | best perplexity, but PubMedQA $-15^*$ (85 % "no"); LoRA answers "no" to everything |
| T61 | evidence data (Section 6) | both arms improve strongly; L2 adds nothing |
| T63 | pair loss (Section 6) | hurts the baseline; the L2 model holds, but not through L2 at test time |

**Table 9:** How the project arrived at the current state. T-numbers are entries in the project's issue list.

Run names used below: **baseline**/**v5** = SmolLM2 alone / + L2 on the old data; **evbase**/**ev5** = the same on the evidence data (T61); **pairbase**/**pair5** = evidence data + pair loss (T63).

# 10. Results

## 10.1. PubMedQA accuracy

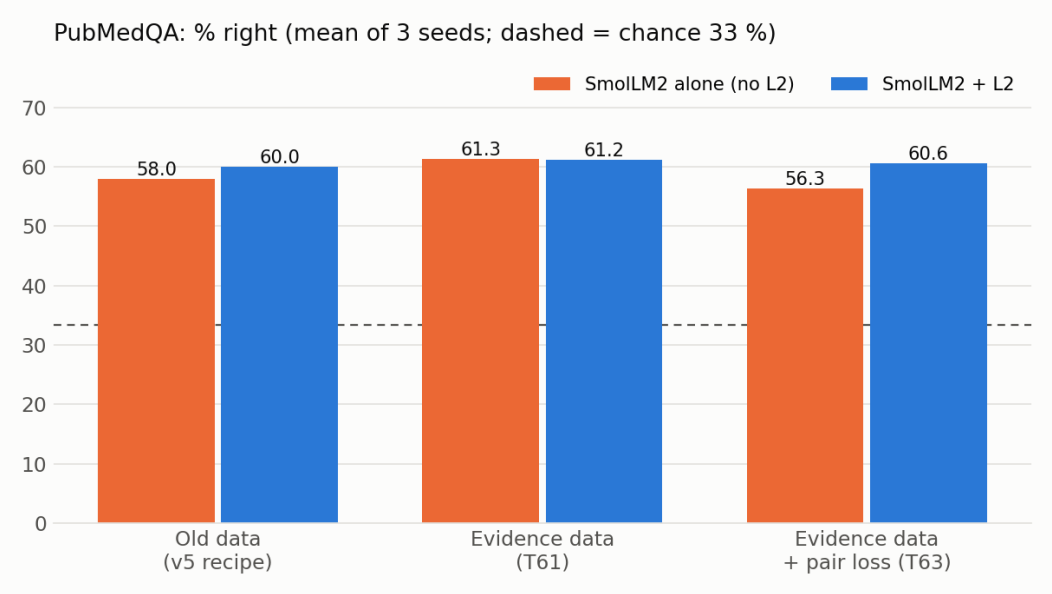



**Figure 10:** PubMedQA accuracy (mean of three seeds). Orange: SmolLM2 alone; blue: SmolLM2 + L2; dashed: chance.

| Training data | SmolLM2 alone | SmolLM2 + L2 | L2 minus alone [95 % CI] |
|---|---|---|---|
| Old data (baseline / v5) | 58.0 | 60.0 | **+2.07** [+0.57, +3.73]* |
| Evidence data (evbase / ev5) | 61.3 | 61.2 | −0.10 [−0.70, +0.47] |
| Evidence + pair loss (pairbase / pair5) | 56.3 | 60.6 | **+4.27** [+3.10, +5.57]* |

**Table 10:** PubMedQA % right.

Against the old baseline (58.0 %), the evidence data gives **+3.30*** without L2 and +3.20* with L2; the 3-benchmark average rises by +1.25* and +1.22* (v5: +0.72*). MedMCQA and MedQA never change by more than ±0.5 points (never starred); they stay near chance for a 135 M model.

## 10.2. Reasoning tests

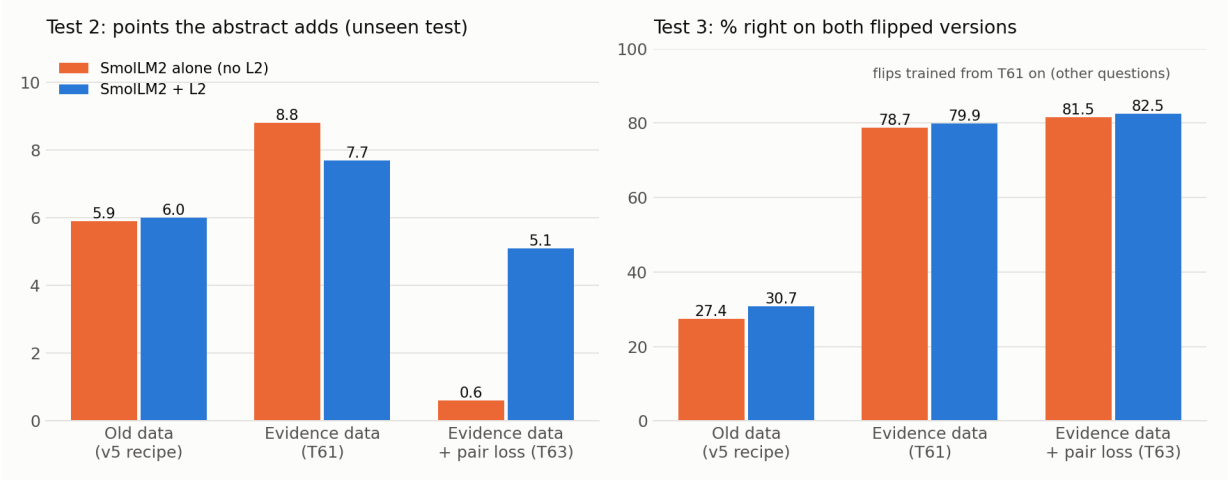



**Figure 11:** Left: test 2, points the abstract adds (unseen). Right: test 3, % right on both flipped versions.

| Model | Uses the abstract (test 2) | Follows flipped evidence (test 3) | Same answer on both versions |
|---|---|---|---|
| baseline / v5 | 5.9 / 6.0 | 27.4 % / 30.7 % | 69.3 % / 65.7 % |
| evbase / ev5 | 8.8 / 7.7 | 78.7 % / 79.9 % | 13.8 % / 12.2 % |
| pairbase / pair5 | 0.6 / 5.1 | 81.5 % / 82.5 % | 14.0 % / 12.8 % |

**Table 11:** Reasoning tests 2 and 3.

Test 3 jumps from about 27 % to about 80 % with the evidence data. This shows that the training worked; it is no longer an unseen test. The pair loss makes SmolLM2 alone almost stop using the abstract (0.6 points).

## 10.3. What L2 adds on the same data

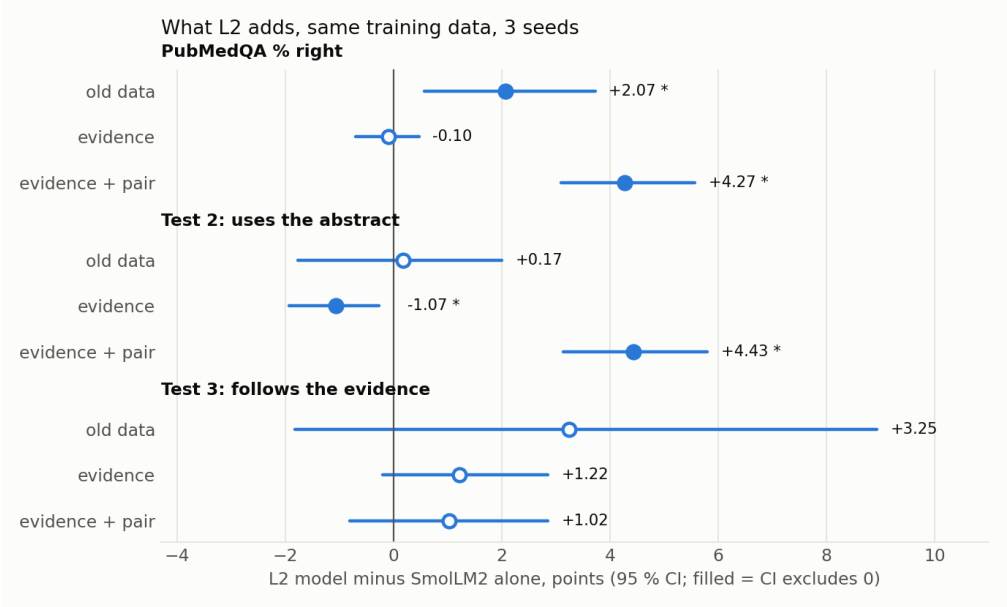



**Figure 12:** L2 minus SmolLM2 alone, same training data, with 95 % CIs. Filled dot: the interval excludes 0. Test 3 is never starred, so the decision rule is never met.

- **Old data:** the gain is a preference (+11.0* more "yes" answers); tests 2 and 3 are not starred.
- **Evidence data:** L2 adds nothing and uses the abstract slightly *less* (−1.07*).
- **Pair loss:** the first starred test-2 gain, on every seed (+4.8* / +5.8* / +2.7*). But it is measured against a baseline the pair loss damaged, test 3 is not starred (+1.02), and the L2-off test shows it is not L2's work.

## 10.4. Switching L2 off at test time

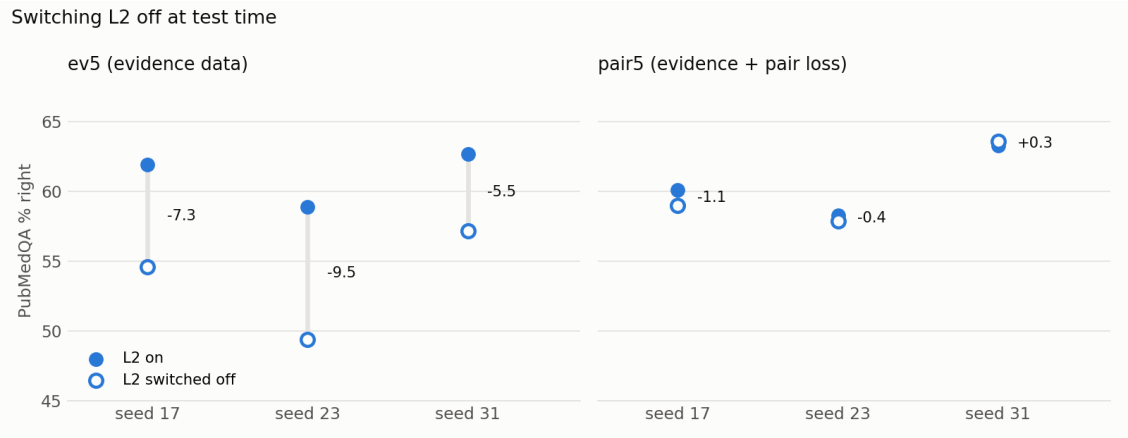



**Figure 13:** PubMedQA per seed with L2 on (filled) and switched off (open).

**ev5 depends on L2** – without it, PubMedQA drops by 5.5–9.5 points – yet it is no better than the model trained without L2: SmolLM2 handed part of its own work to L2. **pair5 does not depend on L2** (−1.1 / −0.4 / +0.3 points; gate 0.015 against 0.027 for ev5). Its lead over pairbase sits in SmolLM2's own weights: training next to L2 kept the backbone from collapsing under the pair loss, but L2 does not compute the answer at test time.

## 10.5. Answer preference

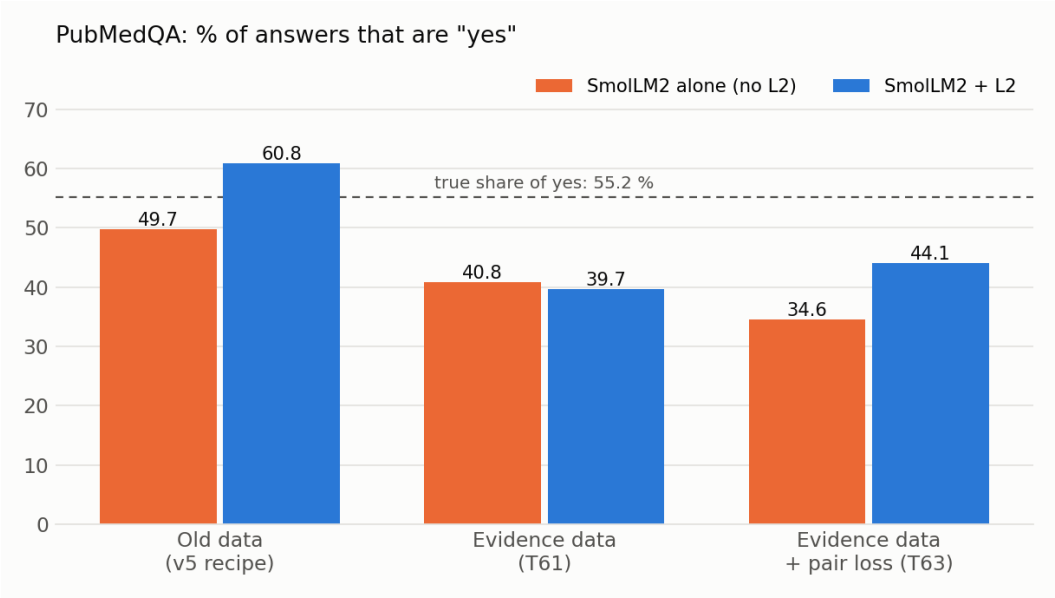



**Figure 14:** Share of "yes" answers on PubMedQA; the dashed line is the true share (55.2 %).

v5 overshoots "yes" (60.8 %). The evidence-trained models under-say it (≈ 40 %): they are right on 87–88 % of the true-no questions and 57–58 % of the true-yes ones. The pair loss pushes SmolLM2 alone further, to 34.6 %.

### 10.6. Text generation

| L2 minus alone, same data | Old data (v5) | Evidence data (ev5) |
|---|---|---|
| MedQuAD perplexity | −0.16 %* | −0.15 %* |
| MedMCQA explanation perplexity | −1.78 %* | −1.46 %* |
| PubMedQA conclusion perplexity | −0.45 %* | +0.84 %* (worse) |
| Generated: explanation ROUGE-L | +0.05 | +0.01 |
| Generated: explanation names the right answer | −0.67 | −0.83 |
| Generated: conclusion ROUGE-L | −0.35 | −0.02 |

**Table 12:** Text measures. Upper block: how well the model predicts reference text; lower block: the text it writes itself.

L2 lowers perplexity slightly (under 2 %, consistently across seeds), but **the text it writes is no better**: no generated-text measure differs significantly. The evidence data has the larger effect: explanation perplexity −5.33 %* without L2 and −6.72 %* with L2 against the old baseline, at the cost of PubMedQA conclusion perplexity (+11.7 / +12.7 %* worse, because less conclusion text is trained). The pair loss makes generated explanations worse: ROUGE-L −1.99* (pairbase) and −3.15* (pair5) against evbase.

## 11. Discussion

### 11.1. Verdict on L2

**Verdict**

**L2 is not useful for this task and is not worth keeping.**

- **Reasoning:** never shown; the pre-registered rule (starred gains on tests 2 and 3) was never met.
- **Accuracy:** its only fair gain (+2.07* on PubMedQA, old data) was a "yes" preference; with the evidence data it adds −0.10 points; MedMCQA and MedQA never moved.
- **Text generation:** perplexity drops by under 2 %, but the written text is no better.
- **Cost:** 19.6 M extra parameters (+14.6 %); the model learns to lean on L2 without beating the model trained without it.

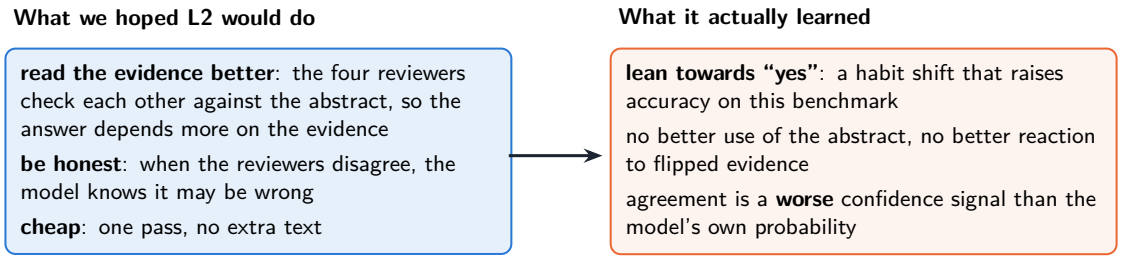



**Figure 15:** What we hoped L2 would do, and what it actually learned.

## 11.2. A likely reason (interpretation)

In v5, L2 only reshapes each token's final state (Section 5); it has no access to evidence that SmolLM2 has not already encoded, and nothing downstream reads its output except the output layer. A module in this position can re-weight the final decision – which is exactly the answer preference we measured – but it has little room to add reasoning. The versions that let L2 read the text and sit inside the network (v3–v4) were not better either, so position alone does not explain the result. This interpretation is not separately tested.

## 11.3. The "yes" preference is a calibration effect

**What was measured** (test 1, `reason_v5.md`, three seeds). The benchmark has 552 "yes", 338 "no" and 110 "maybe" questions.

| PubMedQA, old data | baseline | v5 (+ L2) | difference [95 % CI] |
|---|---|---|---|
| % right, true answer "yes" (552) | 62.1 | 73.1 | +10.99 [+8.88, +12.98]* |
| % right, true answer "no" (338) | 70.1 | 58.3 | −11.83 [−14.50, −9.47]* |
| % right, true answer "maybe" (110) | 0.0 | 0.0 | 0.00 |
| % of answers that are "yes" | 49.7 | 60.8 | +11.03* |
| % right *without* the abstract | 52.1 | 54.0 | +1.90 [+0.73, +3.10]* |
| % right, all questions | 58.0 | 60.0 | +2.07* |
| balanced accuracy, yes/no (mean of the two rows above) | 66.1 | 65.7 | −0.4 |

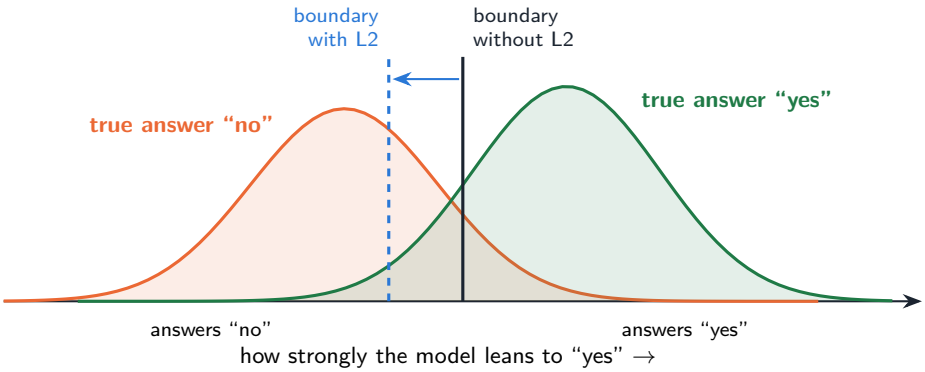
**Table 13:** Where v5's PubMedQA gain comes from. Balanced accuracy is computed from the per-class rows (no separate CI).

**The gain is exactly a threshold shift.** L2 moved the model's yes/no decision boundary towards "yes": it gained 10.99 points on yes-questions and lost 11.83 on no-questions. Because the benchmark has 1.63 times more yes- than no-questions, this trade is a net gain:

$$0.552 \times (+10.99) \;+\; 0.338 \times (-11.83) \;=\; +6.07 - 4.00 \;=\; +2.07 \text{ points.}$$

This is the entire gain. Weighting both classes equally (balanced accuracy), v5 is not better (−0.4). It is a *calibration* change – a different prior over the answers – not a better use of the evidence. Two more observations support this. (i) The same gain appears *without* the abstract (+1.90*), so it does not come from reading the evidence. (ii) It grows with how much the model

relies on L2: with L2 dropped on 30 % of training steps (v5drop), the "yes" share moves by only +2.77* and the gain shrinks to +0.97*.




Moving the boundary left turns some answers into "yes": more yes-questions right (+11.0), more no-questions wrong (−11.8). With 552 yes- and 338 no-questions, accuracy rises by 2.07 points although nothing about reading the evidence changed. (Curves are an illustration; the numbers are measured.)

**Figure 16:** Why accuracy rose without better reasoning: L2 moved the yes/no boundary towards "yes".

**Why it occurred – our explanation (to discuss).**

1. **The decision was never trained, so its threshold is arbitrary.** In the old data the target was only the conclusion text; the yes/no word that the benchmark scores was never a training target (Section 6). Nothing anchors the model's yes-versus-no log-odds, so the baseline lands at 49.7 % "yes" more or less by accident, and any change in the output distribution moves the threshold freely. "Maybe" never appears in training, so it is never answered correctly.

2. **The training text is affirmative.** 92.8 % of the training labels are "yes", and their conclusions are written in affirmative language ("X improves Y"). Fitting this text better means making affirmative continuations more likely, which also raises the probability of the word "yes" after `Answer:`. This is the majority-label bias known from language models [28].

3. **L2's position makes this the easiest thing for it to learn.** L2 sits directly before the output layer and works per token. The cheapest way for such a module to lower the loss on affirmative conclusion text is a near-global shift of the output distribution – which shifts the yes/no log-odds on every question by a similar amount. That is a change of prior, not of reasoning, and it matches both observations above.

4. **When the decision is trained, the effect disappears.** With balanced evidence data (T61), the yes/no decision becomes a training target: ev5 and evbase answer "yes" equally often (39.7 vs 40.8 %) and L2's gain vanishes (−0.10). These models are now miscalibrated in the *other* direction (about 40 % "yes" against a true 55.2 %), because training was 50/50 and the benchmark is not.

**A cheap check we propose (no training).** Fit one yes/no log-odds offset per model on half of the 1,000 PubMedQA questions and score the other half, swapping the halves (contextual calibration [28]). If v5's advantage over the baseline disappears after both are calibrated, the

+2.07 was purely calibration. We also propose reporting balanced accuracy next to accuracy from now on.

## 11.4. Only one backbone: would a larger model change the picture?

**Where we stand.** All L2 results use one backbone, SmolLM2-135M (plus a 50 M from-scratch pilot, v1). We used several *datasets* but only one *model*. The reasons: compute (three seeds for every variant on a shared GPU server), and the step from training from scratch to a pretrained model. The cost is that at 135 M parameters MedMCQA and MedQA stay near chance, so only PubMedQA can show an effect. A negative result at this scale does not show that L2 fails at every scale.

**Why scale could matter.** A larger model encodes more of the evidence in its final state, so there is more for L2 to work with. It is also further from chance on the exam-style benchmarks, so effects become measurable there. On the other hand, the related work [20, 17] and the calibration finding above suggest that what L2 learned is a prior shift, which a larger, better-calibrated model may need even less.

**How L2 grows with the model.** L2's size is $59d^2 + 35d$ parameters for hidden size $d$, so it stays at about 15 % of the backbone:

| Backbone (same family, same tokenizer) | hidden size $d$ | L2 parameters | share of backbone |
|---|---|---|---|
| SmolLM2-135M (used) | 576 | 19.6 M | 14.6 % |
| SmolLM2-360M | 960 | 54.4 M | ≈ 15 % |
| SmolLM2-1.7B | 2048 | 247.5 M | ≈ 15 % |

**Table 14:** L2 size for larger members of the same model family.

**Proposal.** Repeat only the decisive comparison – baseline vs. + L2 on the evidence data, three seeds, all reasoning tests, the L2-off test and the calibration check – on SmolLM2-360M, and on SmolLM2-1.7B if 360M shows any signal. Staying in one family changes only the scale, not the tokenizer or training recipe. Larger models (7 B and up) would need LoRA, and LoRA collapsed to a single answer in T60, so they would be a separate study. This is a decision for the meeting.

## 11.5. Limitations

- **One backbone, small scale.** Only SmolLM2-135M was tested; it stays near chance on MedMCQA and MedQA, so only PubMedQA can show differences (Section 11.4).
- **Calibration.** No model's yes/no threshold was calibrated, and accuracy on an unbalanced benchmark rewards a shifted threshold (Section 11.3).
- **Parameter matching.** The comparison is architecture-matched, not parameter-matched: the L2 arm has 19.6 M more parameters and does more work per token. This favours L2, which strengthens the negative result.

- **Test 3 after T61.** Flipped conclusions are in the training data from T61 on (other questions); test 3 then measures training success, and test 2 is the unseen check.
- **Rule-based flips.** Negations are added or removed by a rule; some flipped sentences may be unnatural. The full list was written out for manual checking.
- **Labels.** `pqa_artificial` labels are generated automatically; the benchmark (`pqa_labeled`) uses expert labels.
- **Seeds.** Three seeds per model; intervals are computed over questions with the same resample in every seed.

## 12. Conclusions

What the thesis can claim:

1. **The data finding.** The yes/no decision was never trained and the data was 92.8 % "yes". Training the decision from balanced and flipped evidence improves PubMedQA (+3.3* points) and the 3-benchmark average (+1.25*), raises following flipped evidence from 27 % to 80 %, and is cheaper to train.
2. **The evaluation method.** Splitting accuracy by the true answer, "uses the abstract", the flipped-evidence test and the L2-off test turned an apparent +2-point "reasoning" gain into an answer preference. Accuracy alone would have hidden it.
3. **A careful negative result for L2**, measured against a same-data baseline on three seeds with confidence intervals.

## 13. Supervisor's comments and points for the meeting

**1. "Only a single model – have you tried a larger LM?"**

No: every L2 result uses SmolLM2-135M. This is a real limitation; at this scale only PubMedQA is informative. We propose repeating the decisive comparison on SmolLM2-360M and, if there is any signal, on SmolLM2-1.7B (same family, L2 stays at about 15 % of the model). See Section 11.4.

**2. "The 'yes' boost looks like a calibration problem – why did it occur?"**

Agreed, and the numbers confirm it: the +2.07 is exactly a threshold shift (+10.99 on yes-questions, −11.83 on no-questions, weighted 552:338), balanced accuracy does not improve, and the gain appears even without the abstract. Our explanation: the yes/no word was never trained, the training text is 92.8 % "yes" and affirmative, and L2's position before the output layer makes a global prior shift the easiest thing for it to learn. A calibration check without training is proposed. See Section 11.3.

**3. "What is the working hypothesis behind L2?"**

That the benefit of considering several alternatives, cross-checking and reconciling them – what self-consistency and debate do in text – can be obtained inside one forward pass on hidden states, making the decision depend more on the evidence and less on priors. It is stated with five testable predictions in Section 2; none of the reasoning predictions was met.

**4. "Related work is missing."**

Added as Section 3: multi-path reasoning in text, latent-space reasoning (including a negative result that matches ours), multi-stream architectures, energy-based refinement, calibration and counterfactual data, and the medical benchmarks. The contribution is a combination of known ingredients, not a new ingredient.

**Further questions from our side.**

1. Should the thesis be framed as a careful negative result for L2, with the data finding, the evaluation method and the calibration analysis as the positive contributions?
2. Which next step: the larger-backbone run (Section 11.4), the calibration check (Section 11.3), or stop experimenting and start writing?
3. A candidate new design, for discussion: let the model write **four candidate answers** (or explanations) and use an L2-style module at the end to **pick the best one** – a learned version of best-of-$N$ / self-consistency [13]. It tests the original idea in text rather than in hidden states, but costs about four times as much at answer time, and on yes/no questions the choice reduces to the calibration problem above; it fits the explanation-writing benchmark better. (Placing L2 at the very start is not promising: L2 works one word at a time, and before the first layers a word's state holds no context yet; earlier placements, v2.1–v4, already helped less than the end.)
4. Final model: the proposal is the evidence-data recipe **without** L2 (evbase) – it scores the same as ev5 with 19.6 M fewer parameters.

# A. Glossary

| Term | Meaning |
|---|---|
| T-numbers | entries in the project issue list (`07-Management/10_Known_Issues.md`); each is one problem or experiment |
| v-numbers | versions of the L2 design (v1 … v5) |
| baseline | SmolLM2 alone (no L2), old data |
| v5 | SmolLM2 + L2, old data; the final L2 model |
| evbase / ev5 | SmolLM2 alone / + L2, evidence data (T61) |
| pairbase / pair5 | the same two with evidence data + pair loss (T63) |
| trajectory | one of L2's four candidate versions of the token's state |
| gate $g$ | 576 numbers deciding how much of L2's output reaches the output layer; 0 = L2 off |
| perplexity | exp of the mean negative log-likelihood per token; lower is better |
| * | the 95 % confidence interval of a difference excludes 0 |

# B. Reproducibility

- Code: repository `Ekdant-LLM`, folder `09-Code` (`src/ekdant/sandbox.py` = L2, `model.py` = gate and wiring, `train_gen.py` = training incl. `--evidence` and `--pair-loss`, `gen_eval.py` = benchmarks and reasoning tests); 346 automated tests.
- Final L2 configuration: `configs/v5_final.json` ($K = 4$, 4 rounds, MLP ratio 4, pre-norm, 8 descent steps, $\eta = 0.0625$, $\rho_{\min} = 0.05$, mean readout, gate cap 1.0, L2 after the final norm).
- Training: `python -m ekdant.train --task gen --batch-size 8 --grad-accum 4 --epochs 1 --ft-epochs 3` plus `--variant baseline|sandbox`, `--seed`, and `--evidence` / `--pair-loss 1.0` where stated.
- Run book with every command and raw result: `run.md`, sections A2, A3o–A3s, A7k–A7n.

# References

[1] L. B. Allal et al. *SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model.* arXiv:2502.02737, 2025.

[2] Q. Jin, B. Dhingra, Z. Liu, W. Cohen, X. Lu. *PubMedQA: A Dataset for Biomedical Research Question Answering.* EMNLP-IJCNLP 2019.

[3] A. Pal, L. K. Umapathi, M. Sankarasubbu. *MedMCQA: A Large-scale Multi-Subject Multi-Choice Dataset for Medical domain Question Answering.* CHIL 2022, PMLR 174.

[4] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, P. Szolovits. *What Disease Does This Patient Have? A Large-Scale Open Domain Question Answering Dataset from Medical Exams.* Applied Sciences 11(14):6421, 2021.

[5] A. Ben Abacha, D. Demner-Fushman. *A Question-Entailment Approach to Question Answering.* BMC Bioinformatics 20:511, 2019.

[6] X. Qiu et al. *PairCFR: Enhancing Model Training on Paired Counterfactually Augmented Data through Contrastive Learning.* ACL 2024.

[7] D. Kaushik, E. Hovy, Z. C. Lipton. *Learning the Difference that Makes a Difference with Counterfactually-Augmented Data.* ICLR 2020.

[8] L. Gao et al. *A Framework for Few-Shot Language Model Evaluation (lm-evaluation-harness).* Zenodo, 2023.

[9] C.-Y. Lin. *ROUGE: A Package for Automatic Evaluation of Summaries.* ACL Workshop Text Summarization Branches Out, 2004.

[10] B. Zhang, R. Sennrich. *Root Mean Square Layer Normalization.* NeurIPS 2019.

[11] B. Efron, R. J. Tibshirani. *An Introduction to the Bootstrap.* Chapman & Hall, 1993.

[12] J. Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* NeurIPS 2022.

[13] X. Wang et al. *Self-Consistency Improves Chain of Thought Reasoning in Language Models.* ICLR 2023.

[14] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch. *Improving Factuality and Reasoning in Language Models through Multiagent Debate.* arXiv:2305.14325, 2023.

[15] T. Liang et al. *Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate.* arXiv:2305.19118, 2023.

[16] J. Wang et al. *Mixture-of-Agents Enhances Large Language Model Capabilities.* ICLR 2025.

[17] A. Smit et al. *Should We Be Going MAD? A Look at Multi-Agent Debate Strategies for LLMs.* ICML 2024.

[18] S. Hao et al. *Training Large Language Models to Reason in a Continuous Latent Space.* arXiv:2412.06769, 2024.

[19] A. Jolicoeur-Martineau. *Less is More: Recursive Reasoning with Tiny Networks.* arXiv:2510.04871, 2025.

[20] P. Rauba, C. Fanconi, M. van der Schaar. *Tiny Autoregressive Recursive Models.* arXiv:2603.08082, 2026.

[21] M. Burtsev, A. Rumshisky. *Multi-Stream Transformers.* arXiv:2107.10342, 2021.

[22] N. Shazeer et al. *Talking-Heads Attention.* arXiv:2003.02436, 2020.

[23] Kerce, Fox. *Dual-Stream Transformer.* arXiv:2603.07461, 2026.

[24] B. Hoover et al. *Energy Transformer.* NeurIPS 2023.

[25] Li et al. *KL-Regularized Reinforcement Learning is Designed to Mode Collapse.* arXiv:2510.20817, 2026.

[26] Yi et al. *Latent Agents.* ACL 2026, arXiv:2604.24881.

[27] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger. *On Calibration of Modern Neural Networks.* ICML 2017.

[28] Z. Zhao, E. Wallace, S. Feng, D. Klein, S. Singh. *Calibrate Before Use: Improving Few-Shot Performance of Language Models.* ICML 2021.